

Turbulence MGD runs: regularised θ , numerics and moment matching

Recap of 23–24 September 2026

Chiara Zelco

24 September 2026

Summary

The goal was to redo the turbulence MGD runs with more particles and to choose the regularisation strength λ of the time-regularised estimator $\Theta_{\text{reg}}(t)$. Looking into how λ acts led to a chain of findings: the block averaging used so far (`n_subsample`) did most of the smoothing, the old dense solver carried a hidden bias, and the statistic basis turned out to be exactly rank-deficient. The solver was replaced, λ can now be chosen *after* a run without re-simulating, and several numerical failure modes were fixed. The most important open problem is scientific: in all turbulence runs, including the July reference batch, **the moments are matched to $\sim 10^{-3}$ for $t < 0.95$ but not near $t = 1$** . Two batches of runs launched on 24 September test the two main suspects. No final λ or final potential set is reported here.

Throughout, [\[measured\]](#) marks something computed on data or in a controlled test, [\[hypothesis\]](#) marks an interpretation not yet tested directly.

1 Starting point: the July reference batch

51 seeds (300–350) of the full potential set (`L_6`, `L_6_psi`, `L_2_lowpass`, `Scalar_psi_gaussianK`, `Scalar_morlet_gaussianK`, fourth-order Mod2 scattering, real and imaginary, $Q = 1$), with $M = 256$ (1024-sample windows coarse-grained twice), $J = 8$, $Q = 3$, $\sigma = 3.5$, $n_t = 40\,000$, $n_1 = 5000$ particles, $\lambda = 5 \cdot 10^{-7}$, `n_subsample` = 200, per-step ridge (`regularization`) = 10^{-2} . SDE time: 10.6–10.7 h per seed on H100 [\[measured\]](#).

2 How λ and `n_subsample` act on Θ_{reg}

The regularised problem (the time coupling follows Guth et al.’s dual score matching) is, in the continuum limit,

$$M(t)\Theta - b(t) = \lambda \frac{d}{dt}(G(t)\dot{\Theta} - c(t)),$$

with $M = \mathbb{E}[\nabla\phi\nabla\phi^\top]$ and $G = \mathbb{E}[\phi\phi^\top]$ on the walkers. The grid spacing does not appear: λ alone sets a smoothing length $\ell \sim \sqrt{\lambda G/M}$.

- `n_subsample` is *not* part of Guth’s method. It was introduced to keep the dense $(nr)^2$ solve in memory, and it averages M, G, b, c over blocks of n_s steps. The block averaging makes it a second smoother (a boxcar of width $\Delta = n_s/n_t$). The effective window is $\approx \max(\Delta, 4\ell)$.
- In the July runs, $\lambda = 5 \cdot 10^{-7}$ was essentially *inactive* for $t < 0.95$: Θ_{reg} equals the block average of the per-step θ_t to within 0.15–0.37 block standard errors; λ only acts for $t > 0.95$, where the Cos schedule makes the steps small (the coupling is λ/dt^2) [\[measured\]](#) (4 seeds).

- The per-step θ_t noise is white (lag-1 autocorrelation of differences -0.455 , vs. -0.5 for white noise), so averaging does reduce variance as expected [measured].
- Each block was labelled with its *first* time although it holds the block average: a half-block time shift [measured] (toy with known θ).

3 Solver: from dense float32 to block-Thomas float64

Problems of the old dense solve [measured] (tests with the actual solver code)

- The ridge $0.01 \cdot I$ was added *after* Jacobi rescaling, i.e. $0.01 \text{diag}(A)$ in original units, which contains the coupling λ/dt^2 . For $\ell/\Delta \gtrsim 2$ it shrinks Θ towards zero (toy with a known θ of RMS 0.87: RMS error 0.82 at $\lambda = 5 \cdot 10^{-5}$, $n_s = 10$, vs. 0.004 without this ridge). So smaller `n_subsample` was *not* better at fixed λ .
- It ran in float32; on ill-conditioned systems with large λ the error reached 640% (the block-Thomas solver in float32 failed similarly).

New solver Block-Thomas elimination, float64, on CPU, $O(nr^2)$ memory, no ridge inside the elimination. The system is symmetric positive definite, so no pivoting across blocks is needed. Matches the exact solution to 10^{-14} – 10^{-8} even at condition number 10^9 [measured]. With `n_subsample = 1` it solves Guth’s system on the full grid (no block averaging). On the Gaussian test case θ is recovered correctly (checked on the cluster). Selected with `--reg-solver thomas`; the old path is unchanged.

4 Choosing λ after the run

λ does not enter the SDE, only the final solve. Runs can therefore save the assembled system (t, M_k, G_k, b_k, c_k ; ~ 25 GB per run) and re-solve it offline for any λ (`--save-reg-system, codes/resolve_theta_reg.py`). The offline solve reproduces the in-run one exactly (difference 0.0) [measured]. On the cluster: 136 s per λ for $r = 235$, $n = 39\,716$ nodes [measured].

Selection rule (`turbulence/lamtune_select.ipynb`) Residual $R = \Theta_{\text{reg}}(\lambda) - \Theta_{\text{reg}}(0)$. While λ only removes noise, block means of R shrink like noise and $\mathbb{E}[z^2] \lesssim 1$; when it removes signal, $\mathbb{E}[z^2] \gg 1$. Choose the largest λ with $\mathbb{E}[z^2] \leq 1$ in every time window. On synthetic data with known $\theta(t)$ this threshold landed on the true RMS-error optimum; a threshold of 2 overshoot by half a decade ($1.6\times$ the optimal error) [measured]. The reference is $\lambda = 0$ of the same solver, not θ_t , whose per-step ridge would appear as spurious structure.

5 Numerical issues found on the turbulence runs

1. **GPU memory.** The order-4 scattering gradient holds ~ 6 complex tensors of shape $(B, 8, 8, 9, 256)$ at once: ~ 6.6 MB per sample [measured]. Batches of 5000 fit an 80 GB H100; a 16 GB V100 needs ≤ 1000 .
2. **Throughput.** $\approx 1.9 \cdot 10^{-4}$ s per particle per step on H100, consistent over three batches [measured]. Hence $n_1^{\text{max}} \approx 18 \text{ h} / (n_t \cdot 1.9 \cdot 10^{-4} \text{ s})$: ~ 8500 at $n_t = 40\,000$; $n_1 = 10\,000$ needs $\sim 21 \text{ h}$, above the 20 h limit.
3. **Dead potentials.** Two of five seeds ($n_1 = 10^4$, $n_t = 33\,000$) crashed at step 32 583 (i.e. $t \rightarrow 1$) with a “singular” per-step Gram. With the ridge added after rescaling, a finite matrix cannot be singular, so the cause is a potential whose gradient is exactly zero on all walkers (0/0 in the rescaling) [measured] (no NaN before the crash). Fix: such a

coefficient is set to 0 for that step only; healthy steps are bit-identical to before. Most likely an emptied region of a `Scalar.*_gaussianK` statistic [\[hypothesis\]](#).

4. **Rank-deficient statistic basis.** With no ridge at all, every offline solve was singular. The scaled M_k has 2–3 eigenvalues at 10^{-16} – 10^{-13} [\[measured\]](#). Traced to `L_6_psi[20–23]`: at $M = 256$, $J = 8$, $Q = 3$ the wavelet channels 21, 22, 23 are *exact copies* (all the single frequency bin 1), channel 20 is a near-copy (a $1.1 \cdot 10^{-4}$ component at bin 2) [\[measured\]](#). Removing exact copies loses nothing ($\theta_a \phi + \theta_b \phi = (\theta_a + \theta_b) \phi$); `--deduplicate_filters` drops them for the channel-list potentials only (25→23 channels). The offline solve uses a ridge $10^{-6} \text{diag}(M_k)$ on the data term. The same happens for the Gaussian bank ($M = 128$, $J = 7$).
5. **Float32 per-step solves.** With near-duplicate statistics, float32 is already $5 \cdot 10^{-4}$ off at ridge 10^{-4} and 66% off at 10^{-8} ; float64 stays at $\sim 2 \cdot 10^{-8}$ [\[measured\]](#). New option `--solve_float64`.
6. **The basis depends on n_1 .** The `Scalar.*_gaussianK` fit keeps a data-dependent number of region statistics: $r = 235$ at $n_1 = 8500$ vs. $r = 291$ at $n_1 = 10^4$ [\[measured\]](#). θ is therefore not comparable coefficient-by-coefficient across n_1 .

6 Moment matching fails near $t = 1$

Relative error between target ($\bar{\phi}_e$) and walker ($\bar{\phi}_p$) moments, as in `plot_moment_matching`:

$$\frac{2 |\bar{\phi}_e - \bar{\phi}_p|}{|\bar{\phi}_e| + |\bar{\phi}_p|},$$

full potential set, 512-sample windows coarse-grained once [\[measured\]](#):

run	mean error per t -window		final step, over moments		
	$t < 0.95$	$[0.95, 1]$	mean	median	p90
$n_1 = 10^4$, $n_t = 33\,000$ (3 seeds)	$1\text{--}1.8 \cdot 10^{-3}$	0.28	1.87	2.00	2.00
$n_1 = 8500$, $n_t = 40\,000$ (5 seeds)	$0.9\text{--}1.5 \cdot 10^{-3}$	0.041	0.28	$1.2 \cdot 10^{-3}$	1.40

The July batch shows the same problem on its per-run moment-matching figures (not yet quantified with the comparison script). A final median of 2.0 is the maximum of this metric.

Interpretation

- Not a particle-count problem: more particles improve the empirical moments; the run with *more* particles but *fewer* steps is the worse one [\[hypothesis\]](#).
- **Suspect 1: the per-step ridge.** 10^{-2} only partly corrects directions in which G is nearly degenerate, and G becomes more degenerate near $t = 1$ (duplicate pairs 3→6); the targets also move fastest there, so partial corrections fall behind [\[hypothesis\]](#).
- **Suspect 2: time resolution near $t = 1$.** Consistent with $n_t = 33\,000$ being worse than 40 000 [\[hypothesis\]](#). A larger `schedule_exponent` puts more of the same steps near $t = 1$ at no cost.
- **Suspect 3: statistics dying near $t = 1$,** which cannot be corrected at all [\[hypothesis\]](#).

7 Runs launched on 24 September

All: `--solve_float64`, `--deduplicate_filters`, $n_t = 40\,000$, seed 900, H100.

label	potentials	ridge	exp.	n_1	purpose
<code>diag_reg*_sched*</code> (4 runs)	full	$10^{-2}, 10^{-4}$	2, 3	2000	clean 2×2
<code>long_<set>_sched3_reg1e-4</code> (4)	all 4 sets	10^{-4}	3	8500	production setting
<code>long_full_sched3_reg1e-2</code>	full	10^{-2}	3	8500	full-size 2×2
<code>long_full_sched2_reg1e-4</code>	full	10^{-4}	2	8500	full-size 2×2

(“exp.” is `schedule_exponent`; ridge is the per-step `regularization`.) The four potential sets are: full; without `L2_lowpass`; without `Scalar_morlet_gaussianK`; without both. The fourth corner of the full-size 2×2 (ridge 10^{-2} , exponent 2) is the existing $n_1 = 8500$ batch. The long runs save their system for the offline choice of λ .

8 Open questions and next steps

1. Which of ridge and time grid fixes $t \rightarrow 1$ (from the 2×2 runs)? If neither does, look at the statistics that die near $t = 1$.
2. λ per potential set from the offline sweep; then choose the minimal set from sample quality (spectrum, structure functions, KL), which does not depend on λ .
3. Route to $n_1 = 10^4$: profile one SDE step. The order-4 gradient computes all $8 \times 8 \times 9$ channel combinations and keeps only the indexed ones; if it dominates the step time, computing only those is the cheapest speed-up. Otherwise, checkpoint the SDE state and chain jobs to lift the 20 h limit.
4. Near-duplicate statistics (channel 20 vs. 21, some `Scalar_psi` region pairs) are kept: they carry a little information. Only their combination is well determined, which matters when interpreting individual θ coefficients.

Code changes (for reference)

- `codes/sde_routines.py`: float64 block-Thomas solver; system saving before the solve; dead-potential guard; optional float64 per-step solves.
- New tools:
 - `codes/resolve_theta_reg.py` (offline λ sweep)
 - `codes/find_duplicate_statistics.py`
 - `turbulence/compare_moment_matching.py`
 - `turbulence/lamtune_select.ipynb`
- `codes/filters_bank.py`, `codes/potentials_builder.py`: `deduplicate_filters`.
- `turbulence/launch/`: one tracked script per experiment plus a shared submission file.
- New run-name tags (`_thomas_nsub<N>`, `_f64`, `_dedupfilt`), so runs with different settings never reload each other. (`regularization` and `schedule_exponent` are still not in the name; the launch scripts use distinct labels.)
- Commits 7075cfb, c7bcc39, 0dad74c.